Exercise Sheet 1

Machine Learning for AIE, WS 2026/27

Mathematical Preliminaries

Team Members		
Last name	First name	Matriculation number
ZENZ	Sebastian	01031159

Contents

1 Task 1: p-Norms **2**
- 1.1 Task 1.1: Euclidean norm . . . 2
- 1.2 Task 1.2: Implementation of `p_norm(x,p)` . . . 3
- 1.3 Task 1.3: Analysis of `p_norm` plots . . . 3

2 Task 2: Linear algebra **4**
- 2.1 Task 2.1: Rank of a matrix . . . 4
- 2.2 Task 2.2: Implementation of `dot(x,y)` and `matmul(A,B)` . . . 5
- 2.3 Task 2.3: Testcases for `dot` and `matmul` . . . 5

3 Task 3: Optimization **6**
- 3.1 Task 3.1: Function analysis . . . 6
- 3.2 Task 3.2: Implementation of `f(x)` and `df(x)` . . . 7
- 3.3 Task 3.3: Gradient descent . . . 7

1 Task 1: p-Norms

1.1 Task 1.1: Euclidean norm

The objective of Task 1.1 is to show that, for $p = 2$, Equation (1) yields the Euclidean norm.

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{D} |x_i|^p\right)^{1/p} \quad (p \geq 1), \qquad \|\mathbf{x}\|_\infty = \max_{i=1,\ldots,D} |x_i|. \tag{1}$$

Tasks:

1. **[2 points]** Show that Equation (1) with $p = 2$ yields the Euclidean norm, i.e., $\|\mathbf{x}\|_2 = \sqrt{x_1^2 + \cdots + x_D^2} = \sqrt{\mathbf{x}^T\mathbf{x}}$. Then compute $\|\mathbf{x}\|_1$, $\|\mathbf{x}\|_2$ and $\|\mathbf{x}\|_\infty$ for $\mathbf{x} = (3, -4)^T$. Show all intermediate steps.

with (1) for $p=2$: $\|x\|_2 = \left(\sum_{i=1}^{D} |x_i|^2\right)^{1/2}$

with $p \in 2\mathbb{Z}$ (even number) $\rightarrow |x_i|^2 = x_i^2$

therefore $\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \ldots x_D^2}$ = Euclidean norm

$x^T x = (x_1, x_2, \ldots x_D) \cdot \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{pmatrix} = x_1x_1 + x_2x_2 + \ldots x_Dx_D$

OR $x_1^2 + x_2^2 + \ldots x_D^2$

$\ldots \sqrt{x^T x} = \sqrt{x_1^2 + x_2^2 + \ldots x_D^2} = \|x\|_2$

Conclusion: (1) for $p=2$ results in:

$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \ldots x_D^2} = \sqrt{x^T x}$

which is the Euclidean norm in $\mathbb{R}^D$.

for $x = \begin{pmatrix} 3 \\ -4 \end{pmatrix}$ $\|x\|_1 = |3| + |-4| = 7$

$\|x\|_2 = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$

$\|x\|_\infty = \max(|3|, |-4|) = 4$

1.2 Task 1.2: Implementation of `p_norm(x,p)`

The function `p_norm(x,p)`, where x is the input vector and p the order of the p-norm, was implemented in the code skeleton. Running Task 1 produces:

```
=== Test 1.1: compare p_norm with np.linalg.norm for a random vector (size=5) for p in
    (1,2,3,inf) ===
p_norm (p = 1) matches np.linalg.norm: True
p_norm (p = 2) matches np.linalg.norm: True
p_norm (p = 3) matches np.linalg.norm: True
p_norm (p = inf) matches np.linalg.norm: True
```

To further validate the implementation of `p_norm`, a test was run for the case $\mathbf{x} = (3, -4)$:

```
=== Test 1.2: evaluate p_norm(x,p) for vector x=(3.0,-4.0) and p in(1,2,inf) ===
p_norm (p = 1) of x: 7.0
p_norm (p = 2) of x: 5.0
p_norm (p = inf) of x: 4.0
```

... which is identical to the solution presented in Task 1.1.

1.3 Task 1.3: Analysis of `p_norm` plots

For $p = 1, 2, \ldots, 20$, the p-norms were computed for the following example vectors:

$$\mathbf{x}_1 = (3, -4)^\top, \qquad \mathbf{x}_2 = (1, 1, 1, 1)^\top, \qquad \mathbf{x}_3 = (0, 2, 0)^\top.$$

Their p-norms are shown in Figure 1. The vector $\mathbf{x}_1$ is the same as the test vector used in Tasks 1.1 and 1.2 and its computed p-norms match the results shown in the plot. As p increases, $\|\mathbf{x}_i\|_p$ converges to $\|\mathbf{x}_i\|_\infty$ for each vector as the largest absolute element value will dominate due to the exponential growth of $|x_i|^p$. The p-norm of $\mathbf{x}_3$ is independent of p, since it has only one non-zero component. For $p \geq 1$ it evalutes to constant:

$$\|\mathbf{x}_3\|_p = (|0|^p + |2|^p + |0|^p)^{1/p} = (2^p)^{1/p} = 2.$$

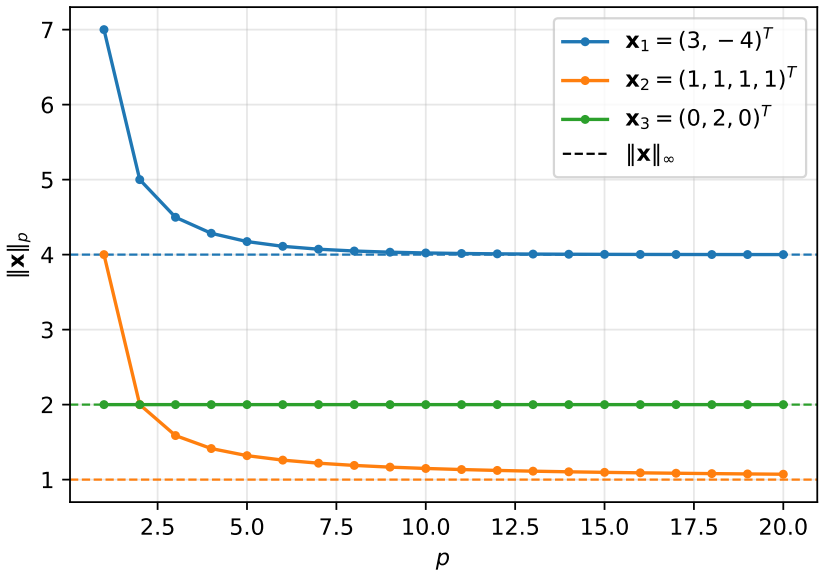


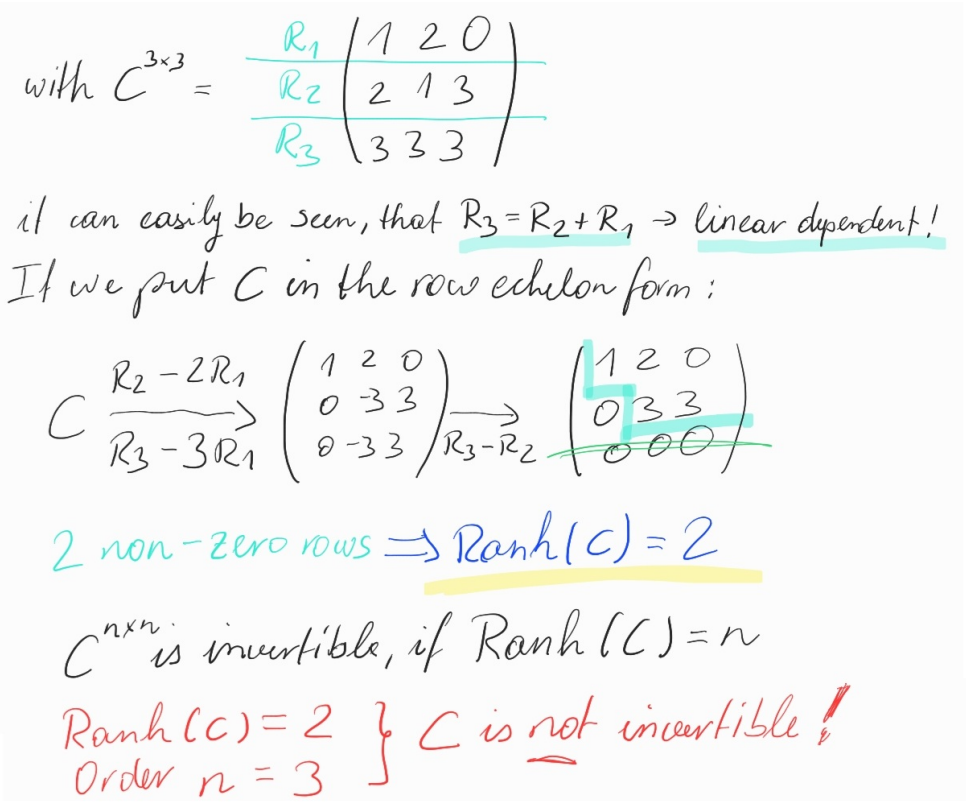
Figure 1: The p-norms of the example vectors for different values of p.

2 Task 2: Linear algebra

2.1 Task 2.1: Rank of a matrix

For Task 2.1, the rank of matrix C is computed using Gaussian elimination. Based on the determined rank, the invertibility of the matrix is then evaluated.

$$C = \begin{pmatrix} 1 & 2 & 0 \\ 2 & 1 & 3 \\ 3 & 3 & 3 \end{pmatrix}$$



Using Gaussian elimination, the matrix C was transformed into row echelon form.
A square matrix is invertible if its rank equals its dimension. Since C is a 3×3 matrix and $\text{rank}(C) = 2 < 3 \implies$ **C is not invertible!**

The result is checked against the output of `np.linalg.matrix_rank`:

```
=== Test 2.1: calculate Rank(C) for given matrix C ===
Matrix C:
 [[1. 2. 0.]
 [2. 1. 3.]
 [3. 3. 3.]]
... evalutes with np.linalg.matrix_rank to: Rank(C) = 2
```

... which is equal to the manually calculated rank(C) in task 2.1.

2.2 Task 2.2: Implementation of `dot(x,y)` and `matmul(A,B)`

Own versions of `np.dot` (dot-product of 2 vectors) and `np.matmul` (matrix-product of 2 matrices) were implemented. The function `matmul(A,B)` calls the function `dot(x,y)` for every entry of the matric C=AB:

```
def matmul(A, B):
    m = np.size(A,0) # m rows
    n = np.size(B,1) # n cols
    C = np.zeros((m,n)) # init m x n zero matrix
    for Ai in range(m):
        for Bi in range(n):
            C[Ai,Bi] = dot(A[Ai,:],B[:,Bi])
    return C
```

2.3 Task 2.3: Testcases for `dot` and `matmul`

Tests for the implementation of `dot(x,y)` and `matmul(A,B)` are run for this task.
The `dot` function was validated by comparing its results with those of `np.dot` with the test vectors:

$$\mathbf{x} = (3, 1, 2.2)^\top, \qquad \mathbf{y} = (4, 0, -1)^\top$$

```
=== Test 2.2: compare dot with np.dot with given vectors
    x=[3. 1. 2.2], y=[ 4. 0. -1.] ===
dot(x, y) = 9.8 matches np.dot: True
```

The `matmul` function was validated by comparing its results with those of `np.matmul` with the given matrices A and B.

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{pmatrix} \qquad B = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

```
=== Test 2.3: compare matmul with np.matmul with examples from the lecture ===
A B =
[[ 9. 12. 15.]
[19. 26. 33.]
[29. 40. 51.]
[39. 54. 69.]]
matches np.matmul: True
```

The `matmul` function was further validated by comparing its results with those of `np.matmul` with random matrices of shape(A) = 3x5 and shape(B) = 5x2.

```
=== Test 2.4: compare matmul with np.matmul for two random matrices of shape (3, 5)
    and (5, 2) ===
A_random B_random =
[[ 4.04680651 1.27937643]
[-0.55145763 -0.06921959]
[-0.1518627 -0.20414412]]
matches np.matmul: True
```

Using `matmul` with two randomly generated 3×3 matrices A and B, we checked the following statements:

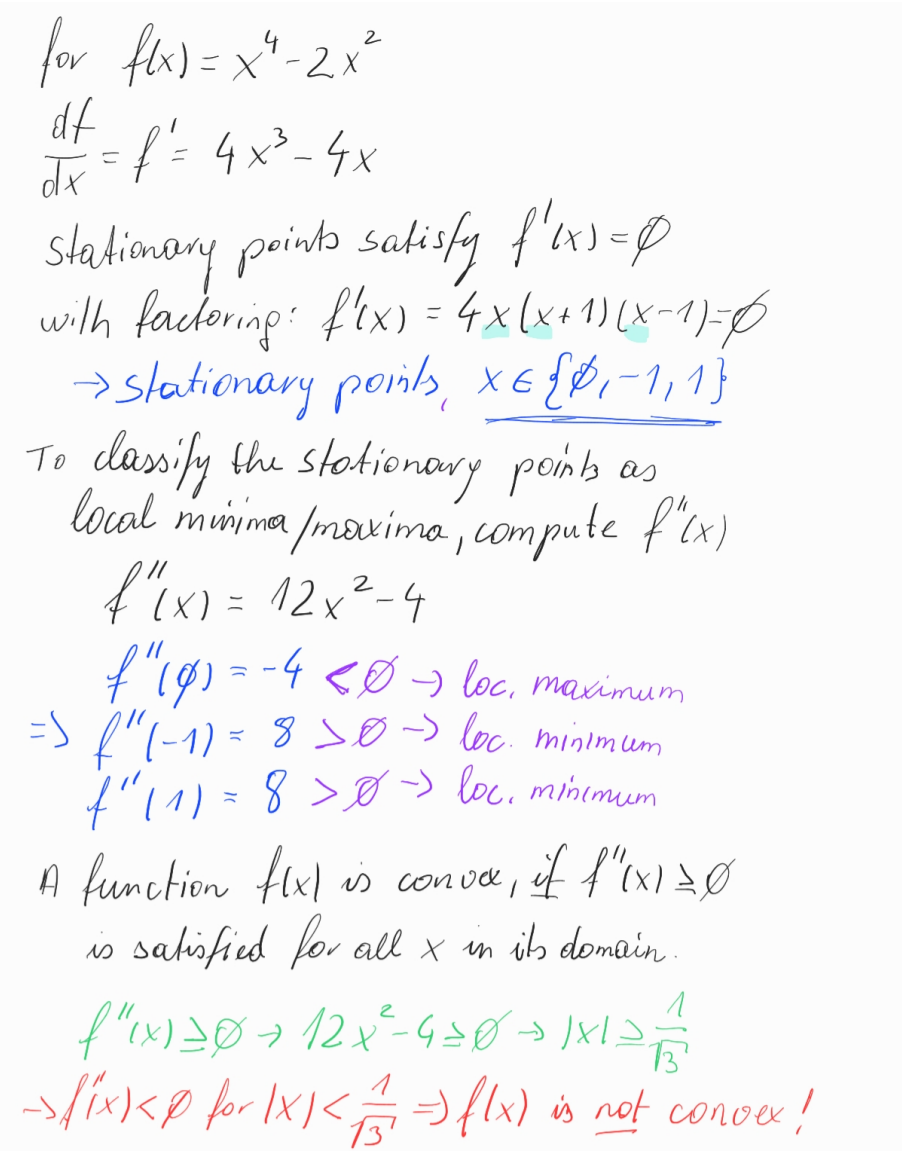
1. $AB = BA$. This statement is false: $AB \neq BA$.
2. $(AB)^\top = B^\top A^\top$. This statement is true.

```
=== Test 2.5: for two random 3x3 matrices A and B, show wheter AB = BA
    and whether (AB)^T = B^T A^T ===
AB equals BA: False
(AB)^T equals B^T A^T: True
```

3 Task 3: Optimization

3.1 Task 3.1: Function analysis

For the function $f(x) = x^4 - 2x^2$ the derivative $f'(x)$ is computed and all stationary points of $f(x)$ are listed and classified as local minima/maxima. The function was also was analyzed to determine whether it is convex.



3.2 Task 3.2: Implementation of `f(x)` and `df(x)`

The functions `f(x)` and `df(x)` were implemented in the code skeleton and the plot of the function and its derivative was created (see Figure 2).
The sign of $f'(x)$ determines whether the function is increasing or decreasing. If $f'(x)>0$, the function is increasing, whereas if $f'(x)<0$, the function is decreasing. Points where $f'(x)=0$ indicate local minima or local maxima.

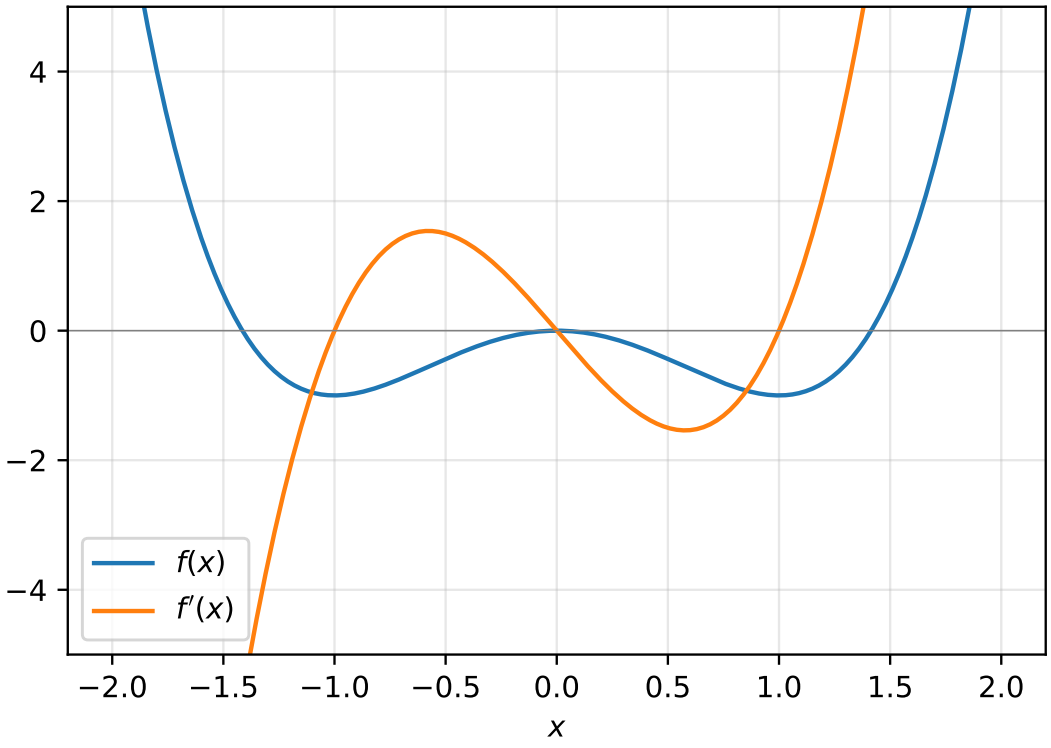


Figure 2: Function f and its derivative f'.

3.3 Task 3.3: Gradient descent

The function `gradient_descent(grad, x0, learning_rate)` was implemented. The basic idea of gradient descent is to iteratively move in the direction of the steepest decrease of the function. The sign of the gradient determines the update direction: when $f'(x)>0$, the function is increasing and the algorithm moves to smaller x-values and vice versa. This process gradually decreases the function value and leads to a local minimum.
The algorithm starts at from an initial point x_0 and follows following iteration steps u ntil the number of maximum iterations is reached or the gradient is within the tolerance ε

$$x^{(t+1)} = x^{(t)} - \eta f'(x^{(t)}), \qquad \text{while } |f'(x^{(t)})| > \varepsilon.$$

Running the code for different starting points x_0 outputs:

```
=== Test 3.1: Number of iterations and final iterate for eta=0.05 for different x0 ===
  x_0: -1.5, n_iter: 28, final_iter: -1.0000000949221568
  x_0: -0.3, n_iter: 37, final_iter: -0.9999999118061864
  x_0: 0.0, n_iter: 0, final_iter: 0.0
  x_0: 0.3, n_iter: 37, final_iter: 0.9999999118061864
  x_0: 1.5, n_iter: 28, final_iter: 1.0000000949221568
```

The plot for the gradient descent for η and different values for the x_0 can be seen in Figure 3. For $x_0 = 0$, the number of iterations equals to zero, as the derivate/gradient of the function evaluates to $f'(0) = 0$ in the first iteration step and the termination condition $|f'(x^{(t)})| \leqslant \varepsilon$ is allready satisfied and a local was never reached by the algorithm.
For the other test cases $x_0 = \{-1.5, -0.3, 0.3, 1.5\}$ their respective local minimum was found after 28 and 37 iterations (termination: gradient within tolerance ε).

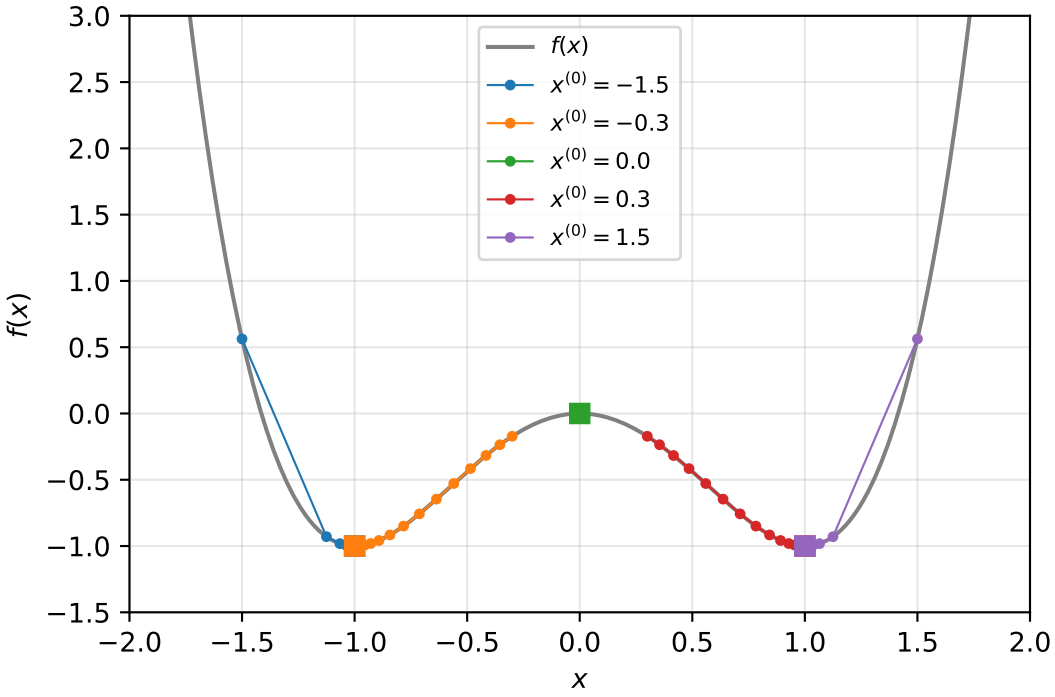


Figure 3: Gradiant descent for different starting points x_0

The learning rate η controls the size of each update step during gradient descent. A small learning rate results in small parameter updates and therefore slow convergence to a minimum. However, if the learning rate is chosen too large, the algorithm may overshoot the minimum, oscillate around it, or fail to converge.

Running `gradient_descent` for the test case x_0=1.5 and different η outputs:

```
=== Test 3.2: Number of iterations and final iterate for x0=1.5 for different etas
    ===
eta: 0.01, n_iter:175, final_iter: 1.00000012108662
eta: 0.05, n_iter:28, final_iter: 1.0000000949221568
eta: 0.15, n_iter:12, final_iter: 0.9999999451609702
eta: 0.3, n_iter:200, final_iter: -1.1395384919254166
```

The results illustrate how the learning rate affects convergence. For the tested learning rates $\eta = \{0.01, 0.05, 0, 15\}$, increasing η (increasing step size) generally leads to faster convergence to the local minimum.
For the test case $\eta = 0.3$ the maximum number of iteration terminated the algorithm before a local minimum was found. The trajectory of the gradient descent algorithm for this test can be found in Figure 4.

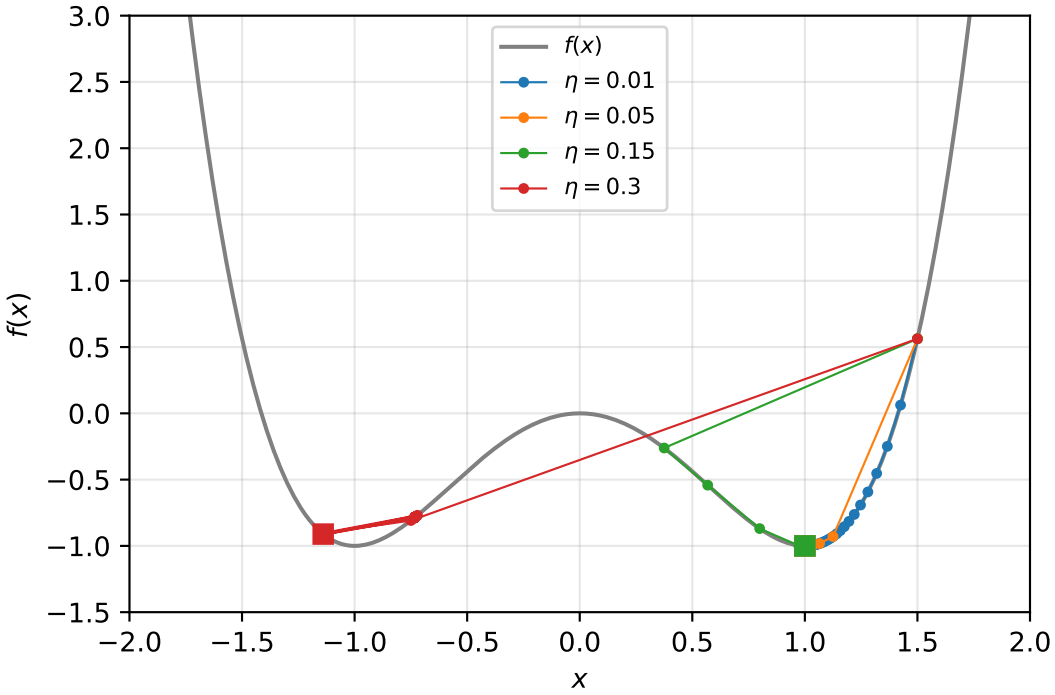


Figure 4: Gradiant descent for different learing rates ε

Further analysis on test case $x_0 = 1.5$, $\eta = 0.3$:

For $\eta = 0.3$, the gradient descent algorithm does not converge. Starting from $x_0 = 1.5$, the first update overshoots the closest local minimum and crosses the central maximum.
As seen in the output of the trajectory and the plot Figure 4 for this case, the first iteration evaluates to $x_1 = -0.75$ and the iterates then settle into a two-cycle/oscilation behaviour instead of approaching a stationary point. The algorithm is then terminated when the maximum number of iterations is reached, before converging to the local minimum.

```
=== Trajectory for x0=1.5 and eta=0.3 ===
trajectories[label]
array([ 1.5 , -0.75 , -1.14375 , -0.72079482, -1.13636604,
       -0.73910004, -1.14152328, -0.72636214, -1.1381206 , -0.73479091,
       -1.14046808, -0.72898614, -1.13889144, -0.73288979, -1.13997128,
...
```